অধ্যায় ২৪

JavaScript থেকে TypeScript — Static Typing, Type Safety এবং Professional Development Workflow

শেখার লক্ষ্য

এই অধ্যায় শেষে পাঠক TypeScript-এর প্রয়োজনীয়তা ব্যাখ্যা করতে, `strict` Configuration ব্যবহার করতে, `unknown` Data Validate করতে, Discriminated Union দিয়ে Success/Error Model করতে, Type-safe Service Layer লিখতে এবং Automated Tests চালাতে পারবেন। Practical Project: **TypeSafe Booking API**।

২৪.১ JavaScript-এর Dynamic Typing থেকে TypeScript

JavaScript Runtime-এ একটি Variable-এর Value পরিবর্তনের সঙ্গে তার Type-ও বদলাতে পারে। এই নমনীয়তা দ্রুত Prototyping-এ সুবিধা দেয়; কিন্তু বড় Codebase-এ ভুল Property Name, ভুল Argument Type, Nullable Data ও API Contract-এর অসামঞ্জস্য ধরতে বেশি সময় লাগে। TypeScript JavaScript-এর ওপর Static Type Checking যোগ করে। TypeScript Compile হলে Type Annotation মুছে JavaScript Output তৈরি হয়। **TypeScript নিজে Runtime Validation করে না।**

```typescript
function total(price: number, quantity: number): number {
  return price * quantity;
}
// total('100', 2); // Compile-time error
```

`tsc` দিয়ে Type-checking করা হয়। TypeScript Type System শক্তিশালী হলেও কিছু ক্ষেত্রে Unsoundness থাকতে পারে— `any`, Type Assertion এবং External Input-এর ক্ষেত্রে সতর্কতা প্রয়োজন।

২৪.২ Project Setup

```terminal
mkdir typesafe-booking-api
cd typesafe-booking-api
npm init -y
npm install --save-dev typescript
npx tsc --init
```

এই বইয়ের Source Folder-এ `package.json`, `tsconfig.json`, `src/domain.ts`, `src/service.ts`, `src/api.ts` এবং `domain.test.mjs` দেওয়া হয়েছে। `module` ও `moduleResolution`-এ `NodeNext` ব্যবহার করা হয়েছে। Import-এ `.js` Extension লেখা হলেও TypeScript Source `.ts` File Resolve করে; Compiled ESM-এ Extension সঠিক থাকে।

২৪.৩ Basic Types ও Inference

```typescript
let customerName: string = 'Amina';
let travelers = 2; // inferred as number
const active: boolean = true;
const destinations: string[] = ['Sylhet', 'Cox's Bazar'];
const scores: Array<number> = [80, 90];
```

`const` Variable Reassignment আটকায়, কিন্তু Object-এর ভেতরের Property স্বয়ংক্রিয়ভাবে Immutable করে না। `readonly` Compile-time Protection দেয়; Runtime-এ Object Freeze করে না।

২৪.৪ `type`, `interface`, Union ও Literal Types

```typescript
type BookingStatus = 'pending' | 'confirmed' | 'cancelled';
type Currency = 'BDT';
interface Package {
  readonly id: string;
  readonly title: string;
  readonly priceBDT: number;
}
```

`interface` Object Contract ও Extensibility-তে সুবিধাজনক। `type` Union, Tuple, Conditional Type ও Composite Type প্রকাশ করতে পারে। একটি Team-এর Consistent Convention সবচেয়ে গুরুত্বপূর্ণ।

২৪.৫ Optional Properties, Null এবং Strict Mode

```typescript
interface Customer {
  id: string;
  email?: string;
}
function displayEmail(customer: Customer): string {
  return customer.email?.toLowerCase() ?? 'No email';
}
```

`strictNullChecks` চালু থাকলে `null` / `undefined` ভুলভাবে ব্যবহার করার সম্ভাবনা কমে। `noUncheckedIndexedAccess` Array Indexing-এর ফলকে Potentially Undefined ধরে; `exactOptionalPropertyTypes` Optional Property-এর Semantics স্পষ্ট করে।

২৪.৬ `unknown` বনাম `any`: Trust Boundary

HTTP Request, JSON File ও `localStorage` থেকে আসা Data **অবিশ্বস্ত**। `JSON.parse()`-এর Result-কে সরাসরি Domain Type হিসেবে Cast করা বিপজ্জনক। `unknown` গ্রহণ করে Runtime Check করুন।

```typescript
function isRecord(value: unknown): value is Record<string, unknown> {
  return typeof value === 'object' && value !== null && !Array.isArray(value);
}
function getCustomerName(input: unknown): string | null {
  if (!isRecord(input)) return null;
  return typeof input.customerName === 'string' ? input.customerName : null;
}
```

Production-এ Zod বা অনুরূপ Schema Validation Library ব্যবহার করা যায়, তবে এখানে Dependency ছাড়াই Validation দেখানো হয়েছে। Type Assertion (`as`) Runtime-এ Value রূপান্তর করে না।

২৪.৭ Discriminated Union দিয়ে Error Handling

```typescript
type Result<T, E> =
  | { ok: true; value: T }
  | { ok: false; error: E };

function divide(a: number, b: number): Result<number, string> {
  return b === 0 ? { ok: false, error: 'Cannot divide by zero' }
                 : { ok: true, value: a / b };
}
const answer = divide(10, 2);
if (answer.ok) console.log(answer.value);
else console.error(answer.error);
```

`ok` Field দেখে Compiler Narrowing করতে পারে। এতে Error Path বাদ পড়ার ঝুঁকি কমে।

২৪.৮ Generics এবং Reusable Abstractions

```typescript
function first<T>(items: readonly T[]): T | undefined {
  return items[0];
}
const n = first([10, 20]); // number | undefined
const s = first(['a', 'b']); // string | undefined
```

Generics দিয়ে Reusable Function Type Information ধরে রাখে। অপ্রয়োজনীয় Complex Generic-এর বদলে পরিষ্কার Domain Type ব্যবহার করুন।

২৪.৯ `keyof`, `Pick`, `Omit`, `Partial`, `Record`

```typescript
interface User { id: string; name: string; role: 'admin' | 'viewer' }
type PublicUser = Pick<User, 'id' | 'name'>;
type NewUser = Omit<User, 'id'>;
type UserPatch = Partial<NewUser>;
const labels: Record<User['role'], string> = {
  admin: 'Administrator', viewer: 'Read-only'
};
```

Utility Types Domain Contract রূপান্তরে সাহায্য করে; এগুলো Runtime Sanitization বা Authorization-এর বিকল্প নয়।

২৪.১০ Exhaustiveness Checking এবং State Machines

```typescript
type Status = 'pending' | 'confirmed' | 'cancelled';
function assertNever(value: never): never {
  throw new Error(`Unexpected: ${String(value)}`);
}
function label(status: Status): string {
  switch (status) {
    case 'pending': return 'Pending';
    case 'confirmed': return 'Confirmed';
    case 'cancelled': return 'Cancelled';
    default: return assertNever(status);
  }
}
```

নতুন Status যোগ করলে Exhaustive Switch Compile-time Feedback দিতে পারে। কিন্তু Business Transition Rules আলাদা Validate করতে হবে—যেমন `cancelled` থেকে `confirmed` -এ ফেরানো নিষিদ্ধ।

২৪.১১ Practical Project — TypeSafe Booking API

এই Project-এর Architecture:

```
Unknown Input
   ↓ Runtime validation (domain.ts)
Typed CreateBookingInput
   ↓ BookingService (service.ts)
InMemoryBookingRepository
   ↓
Typed Booking Result
   ↓
Transport Adapter (api.ts)
```

Features: Package Catalog, Booking Validation, Price Calculation, Duplicate ID Check, Status Transition, Typed Error Codes এবং Automated Tests। Sample Packages ও Prices শুধুই শিক্ষামূলক; বাস্তব Booking বা Payment Processing নেই।

File Responsibilities

- `domain.ts`: Type Definitions, `Result`, Validation, Calculation, Exhaustive Status Label।
- `service.ts`: Dependency-injected Repository, Booking Business Rules।
- `api.ts`: HTTP-like Request Adapter; এটি Network Server নয়।
- `domain.test.ts`: Positive, Negative, Boundary ও State Transition Tests।
- `tsconfig.json`: Strict Compiler Settings।

Build ও Test

```
TERMINAL
cd source-code/chapter-24
npm install
npm run build
npm test
```

`npm run build` TypeScript Source-কে `dist/` Folder-এ JavaScript হিসেবে Emit করবে। `npm test` Build করে Node.js Test Runner চালাবে। Dependency Installation-এর জন্য Internet প্রয়োজন হতে পারে।

Architecture Decisions

Repository Interface: Business Logic Data Storage-এর নির্দিষ্ট Implementation-এর ওপর নির্ভরশীল নয়। **Unknown Input:** API Boundary-তে Validation বাধ্যতামূলক। **Discriminated Result:** Failure Cases Explicit। **Immutable Booking Update:** Status Change-এ Spread দিয়ে নতুন Object তৈরি হয়। **Integer BDT:** Demo-তে পূর্ণ টাকায় হিসাব; Fractional Currency থাকলে Minor Unit Strategy ব্যবহার করুন।

২৪.১২ Common Mistakes

১. `any` দিয়ে Type Error চেপে দেওয়া। ২. `as Booking` লিখে JSON-কে Trusted ধরে নেওয়া। ৩. `strict` বন্ধ রাখা। ৪. `readonly`-কে Deep Runtime Immutability ভাবা। ৫. API Contract আর Database Constraint এক জিনিস মনে করা। ৬. Status Transition Validation বাদ দেওয়া। ৭. Domain Logic সরাসরি UI বা Route Handler-এ লেখা। ৮. Compile Success-কে Full Security Assurance মনে করা।

২৪.১৩ Exercises — ১০টি

১. `Trip` Interface তৈরি করুন, যেখানে `id`, `destination`, `priceBDT` থাকবে।
২. `PaymentStatus` Literal Union তৈরি করুন: `unpaid`, `paid`, `refunded`।
৩. `first<T>()` Function-এর Empty Array Case Handle করুন।
৪. `unknown` Input থেকে Valid Phone String বের করার Type Guard লিখুন।
৫. `Result<number,string>` দিয়ে Discount Calculation করুন।

৬. `Pick` দিয়ে `PublicBooking` Type তৈরি করুন।

৭. `BookingStatus`-এ নতুন `expired` Value যোগ করে Exhaustiveness Error পর্যবেক্ষণ করুন।

৮. Travelers `0`, `21`, `2.5` দিয়ে Validation Test করুন।

৯. Duplicate Booking ID-এর Negative Test লিখুন।

১০. `InMemoryBookingRepository` বদলে File-backed Repository-এর Interface Design করুন (Implementation ঐচ্ছিক)।

Sample Solutions

```typescript
// 1
type Trip = { id: string; destination: string; priceBDT: number };
// 2
type PaymentStatus = 'unpaid' | 'paid' | 'refunded';
// 3
function first<T>(items: readonly T[]): T | undefined { return items[0]; }
// 4
function isPhone(input: unknown): input is string {
  return typeof input === 'string' && /^\+?[0-9]{7,15}$/.test(input);
}
// 5
type Result<T,E> = {ok:true;value:T}|{ok:false;error:E};
function discount(amount:number, percent:number):Result<number,string>{
  if(!Number.isFinite(amount)||amount<0||percent<0||percent>100)
    return {ok:false,error:'Invalid amount or percent'};
  return {ok:true,value:amount*(1-percent/100)};
}
// 6
type Booking = {id:string;customerName:string;totalBDT:number};
type PublicBooking = Pick<Booking,'id'|'totalBDT'>;
// 7: Add 'expired' to the union, then update the exhaustive switch.
// 8–9: See domain.test.mjs for boundary and duplicate tests.
// 10: Implement the BookingRepository interface in a new class.
```

২৪.১৪ Knowledge Check — ১০টি MCQ

১. TypeScript Type Annotation Runtime-এ থাকে? A) সবসময় B) না C) শুধু Browser-এ D) শুধু Node-এ

২. External JSON-এর জন্য নিরাপদ Starting Type? A) any B) never C) unknown D) void

৩. `strictNullChecks` কী করে? A) Null Risk Check B) Network Security C) CSS Validation D) Bundle Compression

৪. `readonly` কি Object Deep Freeze করে? A) হ্যাঁ B) না C) শুধু Array D) শুধু Class

৫. Generic-এর উদ্দেশ্য কী? A) Reusable Typed Logic B) HTML Styling C) Runtime Encryption D) Database Migration

৬. `Pick<User,'id'>` কী? A) Selected Properties Type B) Delete User C) Fetch API D) CSS Selector

৭. Discriminated Union Narrowing-এর ভিত্তি কী? A) Shared Literal Field B) File Name C) Comment D) Folder

৮. `never` কোন কাজে উপযোগী? A) Exhaustiveness Check B) Image Resize C) Cache Expiry D) HTTP Request

৯. `as Booking` কি Runtime Validation? A) হ্যাঁ B) না C) শুধু Test-এ D) শুধু Production-এ

১০. Type-safe Project-এর Production Security-এর জন্য আর কী দরকার? A) Runtime Validation ও Authorization B) শুধু Compile C) শুধু CSS D) শুধু Comments

Answer Key ও Explanations

১. **B** — Compile-এর পরে Type Annotation সাধারণত থাকে না। ২. **C** — `unknown` ব্যবহারে Narrowing বাধ্যতামূলক। ৩. **A** — Nullable Value Access Check করে। ৪. **B** — এটি Static Constraint। ৫. **A** — Type Information ধরে Reuse করা যায়। ৬. **A** — নির্দিষ্ট Properties নির্বাচন করে। ৭. **A** — `ok`-এর মতো Literal Field। ৮. **A** — Unhandled Case ধরতে। ৯. **B** — Type Assertion Runtime Check নয়। ১০. **A** — Compiler Security Boundary নয়।

২৪.১৫ Chapter Summary

TypeScript JavaScript-এর সঙ্গে Type Safety ও Maintainability যোগ করে, কিন্তু Runtime Input, Authentication, Authorization এবং Database Consistency আলাদাভাবে নিশ্চিত করতে হয়। TypeSafe Booking API-এর Typed Domain, Service, Repository ও Adapter Design পরবর্তী **Chapter 25: Node.js Backend, Express, Database ও Authentication**-এর ভিত্তি।